11주차 · 11/12(목)

JavaScript 코어 객체 활용

Array, String, Math 등 주요 객체 다루기 · 함수와 객체

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

11주차

학습 목표

1. 함수를 만들어 반복되는 코드를 묶고 재사용할 수 있다.
2. 화살표 함수를 읽고 쓸 수 있다.
3. Array 메서드(map·filter·find·reduce)를 상황에 맞게 골라 쓸 수 있다.
4. String·Math 등 내장 객체의 주요 메서드를 활용할 수 있다.
5. 객체로 여러 값을 하나로 묶고, 배열과 조합해 자료를 다룰 수 있다.
6. 값이 복사되는 것과 참조가 복사되는 것의 차이를 설명할 수 있다.

11주차

핵심 키워드

function · 화살표 함수 · return · 스코프 · Array · map/filter/find/reduce · String · Math · 객체 · 구조 분해

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

11주차 · 1

함수 — 코드에 이름 붙이기

같은 코드를 세 군데에 복사해 붙였다면, 고칠 일이 생겼을 때 세 군데를 다 고쳐야 합니다.

11주차 · 1

함수 — 코드에 이름 붙이기

1. 하나라도 빠뜨리면 버그입니다.
2. 함수는 그 코드에 이름을 붙여 한 곳에만 두는 것입니다.

함수 만들고 쓰기

```
// 선언
function add(a, b) {      // a, b = 매개변수(parameter)
  return a + b;           // return = 결과를 돌려준다
}

// 호출
const result = add(3, 4);   // 3, 4 = 인자(argument)
console.log(result);        // 7

// return이 없으면 undefined를 돌려줍니다
function greet(name) {
  console.log(`안녕, ${name}`);   // 출력만 하고 돌려주지는 않음
}
const r = greet('한신');    // r은 undefined
```

함수 — 코드에 이름 붙이기

1. console.log() 로 출력하는 것과 return 으로 돌려주는 것은 완전히 다릅니다.
2. 출력은 사람이 보는 것이고, return은 다른 코드가 이어서 쓰는 것입니다.
3. 초보자가 가장 자주 헷갈리는 지점입니다.

요즘 더 많이 쓰는 모양 (1/2)

```
// 함수 선언
function add(a, b) { return a + b; }

// 함수 표현식
const add2 = function (a, b) { return a + b; };

// 화살표 함수
const add3 = (a, b) => { return a + b; };

// 몸통이 return 하나면 중괄호와 return을 생략할 수 있습니다
```

요즘 더 많이 쓰는 모양 (2/2)

```
19  const add4 = (a, b) => a + b;
20
21  // 매개변수가 하나면 괄호도 생략 가능 (팀 관례에 따라 붙이기도 합니다)
22  const double = n => n * 2;
23
24  // 매개변수가 없으면 빈 괄호
25  const now = () => new Date();
26
27  // 객체를 돌려줄 때는 괄호로 감쌉니다 — 안 그러면 중괄호를 몸통으로 봅니다
28  const makeUser = (name) => ({ name, year: 1 });
```

값이 안 넘어왔을 때

```
function greet(name = '손님', greeting = '안녕하세요') {
  return `${greeting}, ${name}님!`;
}

greet();                    // '안녕하세요, 손님님!'
greet('한신');               // '안녕하세요, 한신님!'
greet('한신', '반가워요');   // '반가워요, 한신님!'
```

중괄호 밖에서는 안 보입니다

```
const global = '전역';

function outer() {
  const local = '함수 안';
  console.log(global);   // ✓ 밖의 것은 안에서 보인다
}

console.log(local);      // ✗ ReferenceError: local is not defined

if (true) {
  const blockVar = '블록 안';
}
console.log(blockVar);   // ✗ const/let은 블록 밖에서 안 보인다

// var는 블록을 무시하고 새어 나옵니다. 그래서 쓰지 않습니다.
```

좋은 함수의 조건

정리

1. **이름이 하는 일을 말한다**
   calc() 가 아니라 calculateTotalPrice().
2. **한 가지 일만 한다**
   이름에 "그리고"가 들어가면 두 개로 나눌 때입니다.
3. **밖의 것을 몰래 바꾸지 않는다**
   받은 값으로 계산해 돌려주는 함수가 가장 다루기 쉽습니다.
4. **짧다**
   화면을 넘어가면 나누는 것을 고려하세요.

11주차 · 2

배열 — 순서 있는 값 묶음

기본

```
const fruits = ['사과', '배', '감'];

fruits[0]        // '사과'   ← 0부터 셉니다
fruits[2]        // '감'
fruits[3]        // undefined (없는 자리)
fruits.length    // 3
fruits[fruits.length - 1]   // 마지막 요소
fruits.at(-1)    // '감'  ← 뒤에서 세기 (더 편합니다)
```

추가·제거

```
const a = [1, 2, 3];

a.push(4);        // 뒤에 추가 → [1,2,3,4]
a.pop();          // 뒤에서 빼기 → 4를 돌려주고 [1,2,3]
a.unshift(0);     // 앞에 추가 → [0,1,2,3]
a.shift();        // 앞에서 빼기 → 0을 돌려주고 [1,2,3]

a.splice(1, 1);         // 1번 자리부터 1개 삭제 → [1,3]
a.splice(1, 0, 99);     // 1번 자리에 99 끼워넣기 → [1,99,3]

// 위 메서드들은 원본을 바꿉니다.
```

2.1 배열 메서드 — 여기가 핵심입니다

1. 실무에서 배열을 다룰 때는 for 문보다 아래 메서드를 훨씬 많이 씁니다.

2. 무엇을 하려는지가 코드에 그대로 드러나기 때문입니다.

2.1 배열 메서드 — 여기가 핵심입니다 (1/2)

메서드	하는 일	돌려주는 것
forEach	하나씩 실행	없음(undefined)
map	각각을 변형	같은 길이의 새 배열
filter	조건에 맞는 것만 고르기	짧아진 새 배열
find	조건에 맞는 첫 하나	요소 하나 (없으면 undefined)

2.1 배열 메서드 — 여기가 핵심입니다 (2/2)

메서드	하는 일	돌려주는 것
some	하나라도 맞는가	true / false
every	전부 맞는가	true / false
reduce	전부 합쳐 하나로	결과값 하나
sort	정렬	정렬된 배열 (원본도 바뀜)

map — 각각을 바꿔 새 배열

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);        // [2, 4, 6]
console.log(nums);                            // [1, 2, 3]  ← 원본 그대로

const names = ['한신', '길동'];
const tags = names.map(n => `<li>${n}</li>`);  // HTML 만들기에 자주 씁니다
```

filter — 조건에 맞는 것만

```
const scores = [95, 42, 88, 67];
const passed = scores.filter(s => s >= 60);    // [95, 88, 67]

const students = [
  { name: '한신', year: 1 },
  { name: '길동', year: 2 },
  { name: '영희', year: 1 },
];
const freshmen = students.filter(s => s.year === 1);   // 1학년만
```

find — 첫 하나만

```
const found = students.find(s => s.name === '길동');
// { name: '길동', year: 2 }

// filter는 배열을, find는 요소 하나를 돌려줍니다. 자주 헷갈립니다.
students.filter(s => s.name === '길동')   // [{ ... }]  ← 배열
students.find(s => s.name === '길동')     // { ... }    ← 객체
```

reduce — 하나로 합치기

```
const nums = [10, 20, 30];

const sum = nums.reduce((acc, n) => acc + n, 0);
//                       ↑     ↑           ↑
//                    누적값  현재값      시작값
// 진행: (0,10)→10 · (10,20)→30 · (30,30)→60
console.log(sum);   // 60

const max = nums.reduce((a, b) => (a > b ? a : b));
const total = students.reduce((acc, s) => acc + s.year, 0);
```

sort — 함정이 있습니다

```
const nums = [10, 9, 100, 2];

nums.sort();                        // [10, 100, 2, 9]  ← 문자열로 정렬해서 엉망!
nums.sort((a, b) => a - b);         // [2, 9, 10, 100]  ← 오름차순 ✓
nums.sort((a, b) => b - a);         // [100, 10, 9, 2]  ← 내림차순

// 한국어 이름 정렬
names.sort((a, b) => a.localeCompare(b));

// sort는 원본을 바꿉니다. 원본을 두려면 복사 후 정렬합니다.
const sorted = [...nums].sort((a, b) => a - b);
```

메서드 이어 쓰기 (체이닝)

```
const result = students
  .filter(s => s.year === 1)          // 1학년만 고르고
  .map(s => s.name)                   // 이름만 뽑아
  .sort((a, b) => a.localeCompare(b)) // 정렬
  .join(', ');                        // 문자열로 합치기

console.log(result);   // '영희, 한신'
```

11주차 · 3

객체 — 이름표가 붙은 값 묶음

만들고 꺼내 쓰기

```
const student = {
  name: '김한신',
  year: 1,
  dept: 'AI · SW계열',
  subjects: ['웹프로그래밍', '자료구조'],
  greet: function () {                    // 값이 함수여도 됩니다 (메서드)
    return `안녕, 나는 ${this.name}`;
  },
};

student.name              // '김한신'   ← 점 표기 (기본)
student['name']           // '김한신'   ← 대괄호 표기
const key = 'year';
student[key]              // 1  ← 변수로 접근할 때는 대괄호만 가능

student.greet()           // '안녕, 나는 김한신'
student.grade             // undefined (없는 키)
```

추가·수정·삭제·확인

```
student.email = 'me@example.com';   // 추가
student.year = 2;                   // 수정
delete student.dept;                // 삭제

'name' in student                   // true
Object.keys(student)                // ['name', 'year', ...]
Object.values(student)              // 값들의 배열
Object.entries(student)             // [['name','김한신'], ...]
```

자주 보게 될 문법입니다

```
const { name, year } = student;
console.log(name, year);      // '김한신' 1

// 이름을 바꿔 받기
const { name: studentName } = student;

// 없을 때 기본값
const { grade = 'N/A' } = student;

// 배열도 됩니다
const [first, second] = ['사과', '배'];

// 함수 매개변수에서 바로 분해 (실무에서 아주 흔합니다)
function printStudent({ name, year }) {
  console.log(`${name} (${year}학년)`);
}
printStudent(student);
```

... 세 점

```
const a = [1, 2];
const b = [...a, 3];              // [1, 2, 3]  ← a는 그대로

const base = { name: '한신', year: 1 };
const updated = { ...base, year: 2 };   // { name: '한신', year: 2 }

// 두 배열 합치기
const merged = [...arr1, ...arr2];

// 함수 인자로 펼치기
Math.max(...[3, 1, 4]);   // 4
```

가장 헷갈리는 지점입니다 (1/2)

```
// 기본형 — 값이 복사됩니다
let a = 1;
let b = a;
b = 2;
console.log(a);   // 1  ← a는 그대로

// 참조형 — 주소가 복사됩니다 (같은 것을 가리킴)
const x = { n: 1 };
const y = x;
y.n = 2;
```

가장 헷갈리는 지점입니다 (2/2)

```
19  console.log(x.n);   // 2  x도 바뀝니다!
20
21  // 진짜 복사하려면 펼침을 씁니다
22  const z = { ...x };
23  z.n = 3;
24  console.log(x.n);   // 2  ← 이번엔 안 바뀝니다
25
26  // 펼침은 "얕은 복사"입니다. 안에 또 객체가 있으면 그건 여전히 공유됩니다.
27  const deep = structuredClone(x);   // 깊은 복사가 필요할 때
```

11주차 · 4

배열 + 객체 — 실전 자료 구조

거의 모든 웹 데이터가 이 모양입니다 (1/2)

```
const products = [
  { id: 1, name: '노트북', price: 1200000, category: '전자', stock: 5 },
  { id: 2, name: '마우스', price: 25000,   category: '전자', stock: 0 },
  { id: 3, name: '노트',   price: 3000,    category: '문구', stock: 42 },
];

// 재고 있는 전자제품만
const available = products.filter(p => p.category === '전자' && p.stock > 0);

// 이름만 뽑기
const names = products.map(p => p.name);

// 총 재고 금액
```

거의 모든 웹 데이터가 이 모양입니다 (2/2)

```
const totalValue = products.reduce((sum, p) => sum + p.price * p.stock, 0);

// id로 찾기
const item = products.find(p => p.id === 2);

// 가격순 정렬 (원본 보존)
const cheap = [...products].sort((a, b) => a.price - b.price);

// 화면에 그릴 HTML 만들기
const html = products
  .map(p => `<li>${p.name} — ${p.price.toLocaleString()}원</li>`)
  .join('');
```

객체 ↔ 문자열

```
const obj = { name: '한신', year: 1 };

const text = JSON.stringify(obj);
// '{"name":"한신","year":1}'   ← 문자열

const back = JSON.parse(text);
// { name: '한신', year: 1 }    ← 객체로 복원

// 보기 좋게 (들여쓰기 2칸)
JSON.stringify(obj, null, 2);

// JSON은 키를 반드시 큰따옴표로 감쌉니다. 함수와 undefined는 담기지 않습니다.
```

4.1 JSON — 자료를 주고받는 표준 형식

1. JSON은 12주차에 서버에서 자료를 받아 올 때, 그리고 localStorage 에 저장할 때 반드시 나옵니다.

2. localStorage 는 문자열만 저장하므로 stringify 와 parse 가 늘 짝으로 나옵니다.

11주차 · 5

String — 문자열 객체

자주 쓰는 문자열 메서드 (1/2)

```
const s = '  Hello Web Programming  ';

s.length                    // 25
s.trim()                    // 앞뒤 공백 제거 — 입력값 검사에 필수
s.toUpperCase()             // 대문자로
s.toLowerCase()             // 소문자로
s.includes('Web')           // true — 포함하는가
s.startsWith('  He')        // true
s.endsWith('  ')            // true
s.indexOf('Web')            // 8 — 없으면 -1
```

자주 쓰는 문자열 메서드 (2/2)

```
s.replace('Web', '웹')       // 첫 하나만 바꾼다
s.replaceAll(' ', '_')      // 전부 바꾼다
s.split(' ')                // 공백으로 잘라 배열로
s.slice(2, 7)               // 일부 잘라내기
s.padStart(3, '0')          // '007' 처럼 앞을 채우기
s.repeat(2)                 // 두 번 반복

// 문자열은 바뀌지 않습니다(immutable).
//   s.toUpperCase()는 새 문자열을 돌려줄 뿐 s 자체는 그대로입니다.
```

11주차 · 6

Math — 수학 객체

Math는 new 없이 바로 씁니다

```
Math.round(3.6)      // 4    반올림
Math.floor(3.9)      // 3    내림
Math.ceil(3.1)       // 4    올림
Math.trunc(-3.9)     // -3   소수점 버리기(음수 주의: floor는 -4)
Math.abs(-5)         // 5    절댓값
Math.max(1, 5, 3)    // 5
Math.min(1, 5, 3)    // 1
Math.pow(2, 10)      // 1024  (2 ** 10 과 같음)
Math.sqrt(16)        // 4
Math.PI              // 3.141592653589793
Math.random()        // 0 이상 1 미만 난수

// 1~n 무작위 정수 — 자주 쓰는 관용구
function randInt(n) { return Math.floor(Math.random() * n) + 1; }

// 배열에서 무작위로 하나 고르기
const arr = ['가위', '바위', '보'];
arr[Math.floor(Math.random() * arr.length)]
```

Math — 수학 객체

1. Math.max(...arr) 처럼 펼침(spread) 과 함께 쓰면 배열의 최댓값을 구할 수 있습니다.
2. Math.max(arr) 는 배열을 숫자로 못 바꿔 NaN 이 나옵니다 — 자주 하는 실수입니다.

11주차 · 7

Number — 숫자 다루기

변환과 검사

```
Number('42')              // 42
Number('42px')            // NaN
parseInt('42px')          // 42    앞에서부터 숫자만
parseFloat('3.14m')       // 3.14

Number.isNaN(x)           // NaN인가 (전역 isNaN보다 정확)
Number.isInteger(4.0)     // true

(3.14159).toFixed(2)      // '3.14'  ← 문자열로 돌려줍니다
(1234567).toLocaleString('ko-KR')   // '1,234,567'

console.log(0.1 + 0.2);  // 0.30000000000000004  ← 컴퓨터 소수 표현의 한계
(0.1 + 0.2).toFixed(2);  // '0.30'
```

11주차

실습

이 시간의 실습 4개 — 학습사이트의 코드 실습장에서 직접 치고 실행합니다

실습 1 · 기본

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 원의 반지름을 받아 넓이를 돌려주는 함수 circleArea를 만들고, 반지름 5의 결과를 소수점 둘째 자리까지 출력하세요.

힌트

넓이는 Math.PI * r * r 입니다. 소수점은 toFixed(2)로 자릅니다. 출력이 아니라 return으로 돌려주는 것이...

실습 1 모범답안

모범답안

```
function circleArea(r) {
  return Math.PI * r * r;
}

console.log(circleArea(5).toFixed(2));   // 78.54
```

실습 2 · 기본

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 점수 배열 [95, 42, 88, 67, 55]에서 60점 이상만 골라 새 배열을 만들고, 그 개수와 평균을 출력하세요.

힌트

filter로 고르고, reduce로 합을 구한 뒤 length로 나눕니다.

실습 2 모범답안

모범답안

```
const scores = [95, 42, 88, 67, 55];

const passed = scores.filter(s => s >= 60);
const sum = passed.reduce((acc, s) => acc + s, 0);
const avg = sum / passed.length;

console.log('합격:', passed);
console.log('인원:', passed.length);
console.log('평균:', avg.toFixed(1));
```

실습 3 · 응용

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 학생 객체 배열을 만들고(이름·학년·점수 3명 이상), 1학년만 골라 이름만 뽑은 뒤 쉼표로 이어 출력하세요.

힌트

filter → map → join 을 이어서 씁니다. 체이닝이라고 부릅니다.

실습 3 모범답안

모범답안

```
const students = [
  { name: '김한신', year: 1, score: 88 },
  { name: '이길동', year: 2, score: 74 },
  { name: '박영희', year: 1, score: 92 },
];

const freshmen = students
  .filter(s => s.year === 1)
  .map(s => s.name)
  .join(', ');

console.log('1학년:', freshmen);
```

실습 4 · 심화

실습

JS · 학습사이트 실습장에서 작성 · 「확인」으로 채점

> 상품 배열에서 ① 총 재고 금액 ② 재고가 0인 상품 이름 ③ 가장 비싼 상품을 구해 출력하세요. 상품은 id·name·price·stock을 가집니다.

힌트

reduce로 합계, filter로 품절, sort나 reduce로 최고가를 구합니다. 원본을 정렬하지 않도록 복사해서…

실습 4 모범답안

모범답안

```
const products = [
  { id: 1, name: '노트북', price: 1200000, stock: 5 },
  { id: 2, name: '마우스', price: 25000, stock: 0 },
  { id: 3, name: '노트', price: 3000, stock: 42 },
];

// ① 총 재고 금액
const totalValue = products.reduce((sum, p) => sum + p.price * p.stock, 0);
console.log('총 재고 금액:', totalValue.toLocaleString() + '원');

// ② 품절 상품
const soldOut = products.filter(p => p.stock === 0).map(p => p.name);
console.log('품절:', soldOut);

// ③ 가장 비싼 상품 (원본을 건드리지 않으려고 복사 후 정렬)
const priciest = [...products].sort((a, b) => b.price - a.price)[0];
console.log('최고가:', priciest.name);
```

11주차 마무리

11주차 마무리

JavaScript 코어 객체 활용 — 이 시간에 다룬 것을 학습사이트에서 다시 확인합니다

- ✓ 학습사이트 주차 페이지의 절마다 「이해했어요 · 실습했어요 · 어려워요」 표시
- ✓ 실습 4개를 실습장에서 「확인」 까지 — 「실습했어요」 체크
- ✓ 퀴즈 9 「퀴즈 9 — JavaScript 코어 객체」 응시 (성적 미반영 · 학습 확인용)
- ✓ 막히면 「질문있어요」 또는 패들릿에 — 30분 넘게 혼자 막혀 있지 않기

다음 시간

12주차 · HTML5 DOM 활용